

Case Study 1: Investigating Harassment Email Traffic With Wireshark

OJE DOMINION MAYOWA

FWSD2511507

Computer and Digital Forensics

MR. Aminu Idris

5th October 2026

LAB OVERVIEW & EXECUTIVE SUMMARY

In this case study, I investigated a harassment complaint using nitroba.pcap, a packet capture from a university residence network. My aim was to find out whether the traffic could show where a threatening message sent through the anonymous service willselfdestruct.com came from. I preserved the original capture as read-only, recorded SHA-256 hashes for it and my working copy, and carried out all analysis on the working copy using tshark.

I began by identifying the client and the service. DNS and HTTP traffic showed that the client at 192.168.15.4 looked up and then visited www.willselfdestruct.com (69.25.94.22). At 06:04:24 UTC on 22 July 2008, that client sent an HTTP POST (frame 83601, stream 1707). Following the TCP stream, I recovered the form data: the message was addressed to lilytuckrige@yahoo.com, had the subject "you can't find us", and carried a threatening body. The server accepted it with a 302 redirect to a confirmation page.

Next, I examined the device behind the traffic. All 34,554 frames from 192.168.15.4 carried the MAC address 00:17:f2:e2:c0:ce, so the POST can be tied to one network interface and not just to an IP address. I then searched that device's cookies for identity evidence. After ruling out advertising and shopping cookies, I found that the device carried the cookie jcoachj@gmail.com in 109 frames, and no other device in the capture sent it. Gmail activity from this device spanned the time of the POST. The account name is consistent with Johnny Coach, who is listed on the Chem 109 roster.

My findings support high confidence that the message was sent from this device, and moderate confidence that the device was associated with the Gmail account. I treat Johnny Coach as an investigative lead and not as a proven sender, because the evidence identifies a device and a browser session, not the person using it. The name match is an inference, MAC addresses can be spoofed, and the server's clock differed from the capture timestamps by about 1 hour 20 minutes, so I used the PCAP timestamps for my timeline. The capture also covers only unencrypted traffic seen at one point on the network.

To go further, I would recommend confirming device ownership through network registration records, interviewing the account holder, and examining the device itself.

OBJECTIVES

1. To trace the harassing message to its source device by analysing the packet capture (IP, MAC and message content).
2. To assess what the evidence shows about who was responsible, separating facts from inferences and limitations.

TOOLS

- Kali Linux (VMware Workstation)
- capinfos
- sha256sum
- wget
- Linux shell utilities (cp, chmod, tee, sort, uniq, grep, less, date)
- Chem 109 roster and scenario materials
- Wireshark, tshark
- Microsoft Word

METHODOLOGY

The First step in carrying out this case study was to go through the case study documents thoroughly to understand all that is involved. Then the next step was to create a working environment for the Case study.

Step 1: Lab Folder Structure and Evidence Preparation

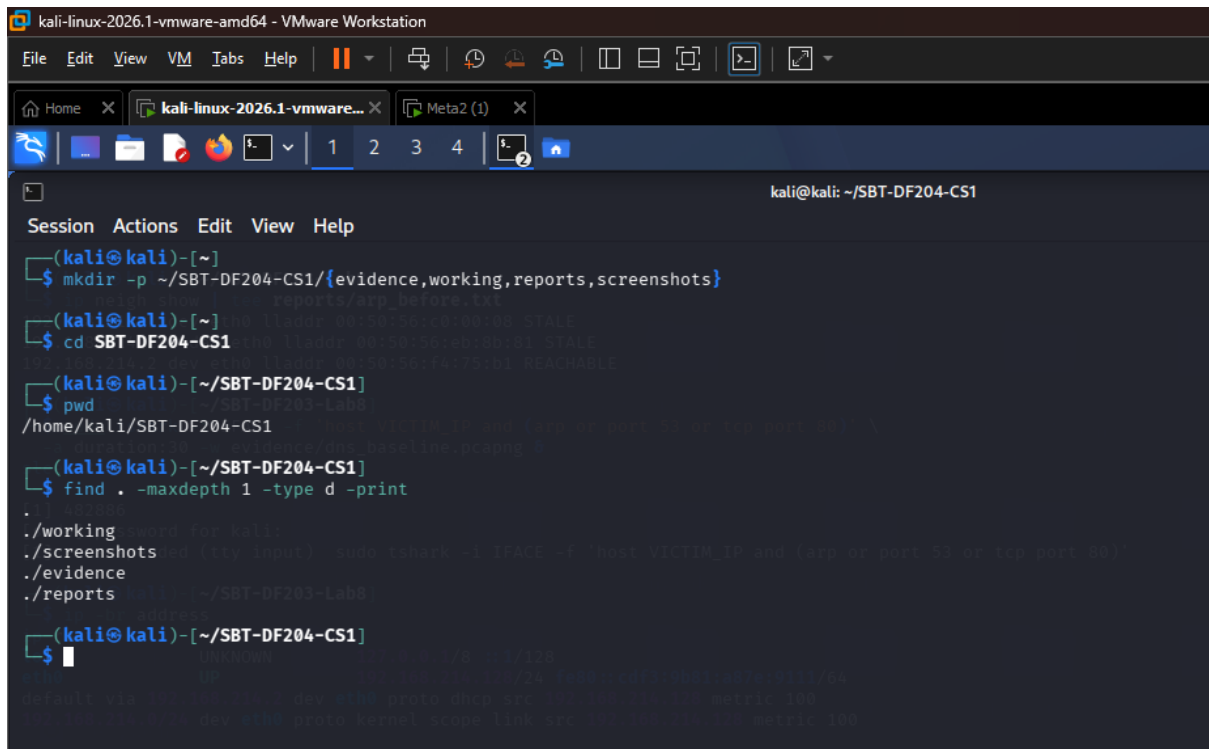


Fig 1: Lab Folder Created and associated folders

Now I updated the lab environment and downloaded every needed dependency or tool to carry out the case study.

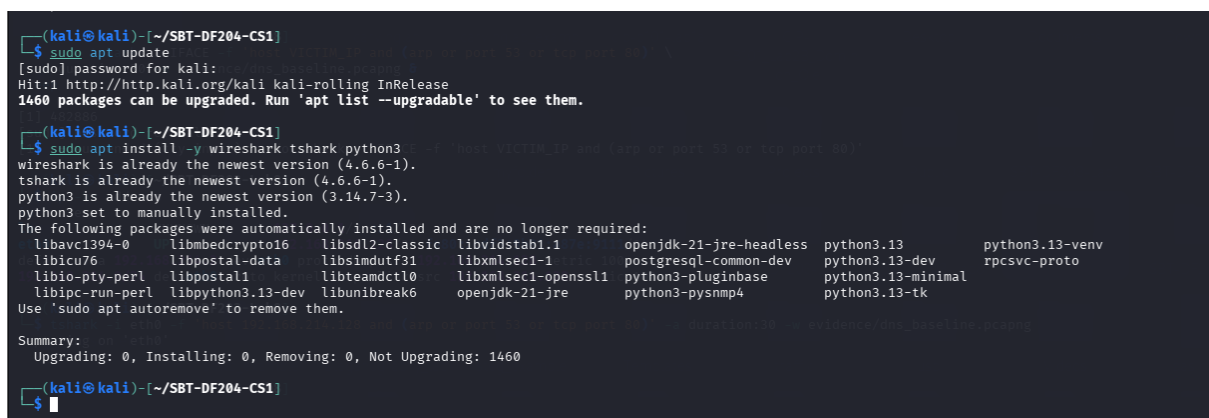


Fig 2: Updating the Lab Environment and Downloading the tools needed

Next, I downloaded the source evidence from the case study document and following every possible authorization by the academy.

```
(kali㉿kali)-[~/SBT-DF204-CS1]
└─$ wget -O evidence/nitroba.pcap 'https://digitalcorpora.s3.amazonaws.com/corpora/scenarios/2008-nitroba/nitroba.pcap'
--2026-10-04 22:08:00-- https://digitalcorpora.s3.amazonaws.com/corpora/scenarios/2008-nitroba/nitroba.pcap
Resolving digitalcorpora.s3.amazonaws.com (digitalcorpora.s3.amazonaws.com)... 52.92.236.169, 16.15.45.194, 52.218.233.123, ...
Connecting to digitalcorpora.s3.amazonaws.com (digitalcorpora.s3.amazonaws.com)|52.92.236.169|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 56180821 (54M) [binary/octet-stream]
Saving to: 'evidence/nitroba.pcap'

evidence/nitroba.pcap 56180821 100%[=====] 53.58M 39.9KB/s in 23m 57s

2026-10-04 22:32:25 (38.2 KB/s) - 'evidence/nitroba.pcap' saved [56180821/56180821]

(kali㉿kali)-[~/SBT-DF204-CS1]
└─$
```

Fig 3: Evidence Downloaded

Now I confirm the evidence file has been saved successfully in the lab environment,

```
(kali㉿kali)-[~/SBT-DF204-CS1]
└─$ ls -l evidence/nitroba.pcap | tee reports/file_size.txt
-rw-rw-r-- 1 kali kali 56180821 Nov 22 2020 evidence/nitroba.pcap

(kali㉿kali)-[~/SBT-DF204-CS1]
└─$
```

Fig 4: Confirmed the Source file is in the Lab environment

Then we copy the Evidence file from the Evidence folder using the **cp** command into a working Folder so we can preserve the Source file and work with a copy of the original.

```
(kali㉿kali)-[~/SBT-DF204-CS1]
└─$ cp --preserve=timestamps evidence/nitroba.pcap working/nitroba_working.pcap

(kali㉿kali)-[~/SBT-DF204-CS1]
└─$
```

Fig 5: Copy of the Original Source File

The Original hash values for the evidence is seen as displayed on the scenario webpage

Hash values for nitroba.pcap:

Algorithm	Value
MD5	9981827f11968773ff815e39f5458ec8
SHA1	65656392412add15f93f8585197a8998aaeb50a1
SHA256	2b77a9eafc1d6af163d1ba793c96dbccac04e6befdf1a0b01f8c67553ec2fb

Fig 6: Original Hash Values of the Source File

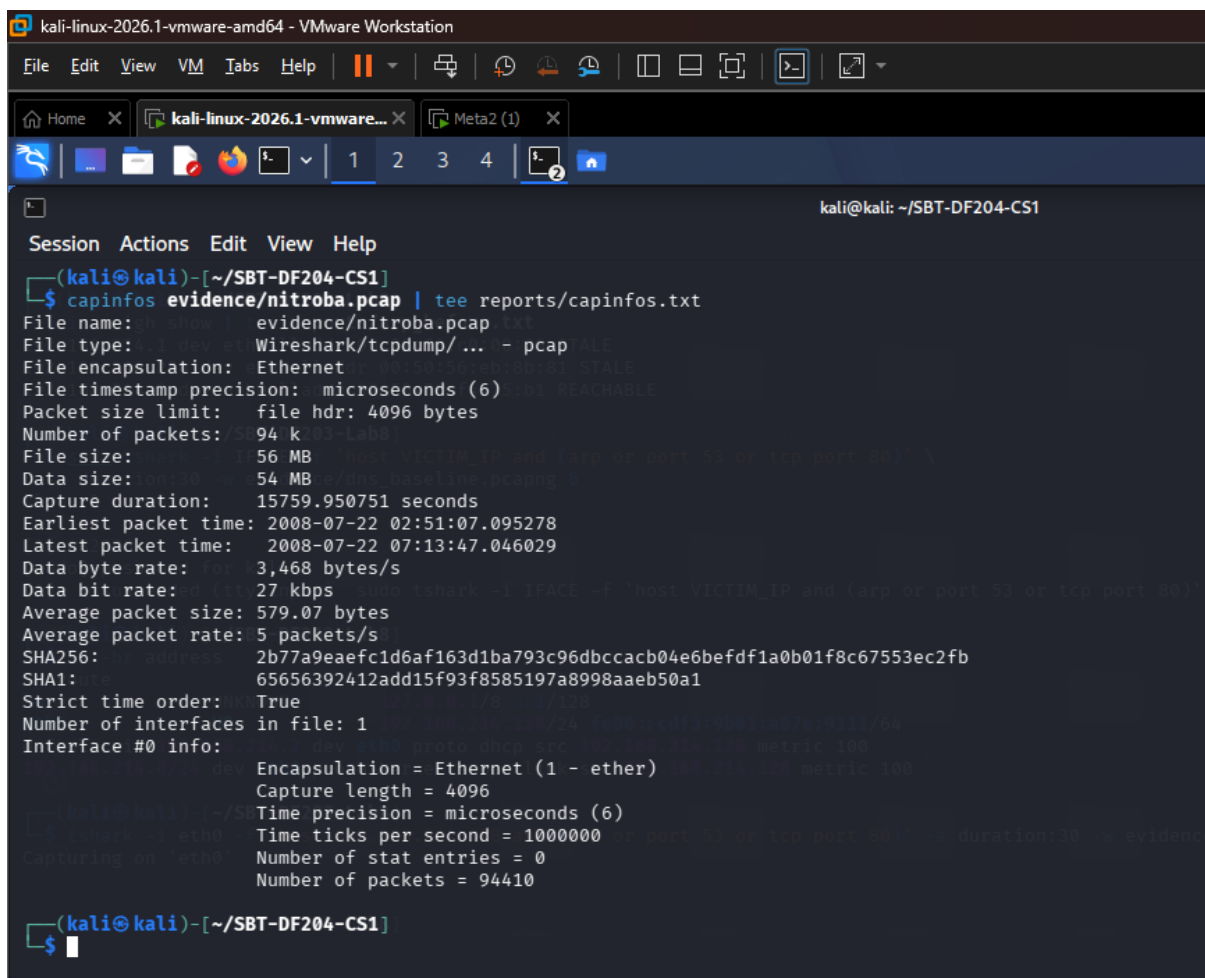
```
(kali㉿kali)-[~/SBT-DF204-CS1]
$ sha256sum evidence/nitroba.pcap working/nitroba_working.pcap | tee reports/hashes.txt
2b77a9eaefc1d6af163d1ba793c96dbccac04e6befdf1a0b01f8c67553ec2fb evidence/nitroba.pcap
2b77a9eaefc1d6af163d1ba793c96dbccac04e6befdf1a0b01f8c67553ec2fb working/nitroba_working.pcap

(kali㉿kali)-[~/SBT-DF204-CS1]
$
```

Fig 7: Hash Values from Kali Linux

This is the sha256sum hash value generated from kali Linux for both the original and the Copy and it has the same value as the one from the original scenario site.

Then we get the information of the captured packet *nitroba.pcap*



```
kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2026.1-vmware... Meta2 (1)
kali@kali: ~/SBT-DF204-CS1
Session Actions Edit View Help
(kali㉿kali)-[~/SBT-DF204-CS1]
$ capinfos evidence/nitroba.pcap | tee reports/capinfos.txt
File name: evidence/nitroba.pcap
File type: Wireshark/tcpdump/... - pcap
File encapsulation: Ethernet
File timestamp precision: microseconds (6)
Packet size limit: file hdr: 4096 bytes
Number of packets: 94 k
File size: 56 MB
Data size: 54 MB
Capture duration: 15759.950751 seconds
Earliest packet time: 2008-07-22 02:51:07.095278
Latest packet time: 2008-07-22 07:13:47.046029
Data byte rate: 3,468 bytes/s
Data bit rate: 27 kbps
Average packet size: 579.07 bytes
Average packet rate: 5 packets/s
SHA256: 2b77a9eaefc1d6af163d1ba793c96dbccac04e6befdf1a0b01f8c67553ec2fb
SHA1: 65656392412add15f93f8585197a8998aaeb50a1
Strict time order: True
Number of interfaces: in file: 1
Interface #0 info: Encapsulation = Ethernet (1 - ether)
Capture length = 4096
Time precision = microseconds (6)
Time ticks per second = 1000000
Number of stat entries = 0
Number of packets = 94410

(kali㉿kali)-[~/SBT-DF204-CS1]
$
```

Fig 8: Information on the Captured Packet

Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF204-CS1-Oje-Dominion
Trainee name	OJE DOMINION MAYOWA
Date and time started	03/10/2026, 3:00pm
Evidence file name(s)	<i>nitroba.pcap</i>
Source or generation method	https://digitalcorpora.s3.amazonaws.com/corpora/scenarios/2008-nitroba/nitroba.pcap
File Size	56MB
Original SHA-256	2b77a9eaefc1d6af163d1ba793c96dbccacb04e6befdf1a0b01f8c67553ec2fb
Working-copy SHA-256	2b77a9eaefc1d6af163d1ba793c96dbccacb04e6befdf1a0b01f8c67553ec2fb
Analysis workstation/VM	Kali Linux VM
Notes on any changes	

Step 2: Getting the Client and the Service

Now we want to use **tshark** to run analysis on our captured packet to be able to decipher the client that was sending it and the client it was going to.

```
(kali@kali)-[~/SBT-DF204-CS1]
$ tshark -r working/nitroba_working.pcap -Y 'http.request.method=="GET" && http.host contains "willselfdestruct"' \
-T fields -e frame.number -e frame.time_utc -e ip.src -e ip.dst -e http.host -e http.request.uri
82936  2008-07-22T06:03:43.825871000Z  192.168.15.4  69.25.94.22  www.willselfdestruct.com  /secure/submit
82985  2008-07-22T06:03:44.125949000Z  192.168.15.4  69.25.94.22  www.willselfdestruct.com  /images/spacer.gif
83025  2008-07-22T06:03:44.334254000Z  192.168.15.4  69.25.94.22  www.willselfdestruct.com  /images/sm-logo.gif
83037  2008-07-22T06:03:44.374583000Z  192.168.15.4  69.25.94.22  www.willselfdestruct.com  /images/warning-home.gif
83072  2008-07-22T06:03:44.530352000Z  192.168.15.4  69.25.94.22  www.willselfdestruct.com  /images/body-bk.gif
83087  2008-07-22T06:03:44.599079000Z  192.168.15.4  69.25.94.22  www.willselfdestruct.com  /images/btn-send.gif
83162  2008-07-22T06:03:44.852265000Z  192.168.15.4  69.25.94.22  www.willselfdestruct.com  /images/bridge_small.gif
83614  2008-07-22T06:04:24.564165000Z  192.168.15.4  69.25.94.22  www.willselfdestruct.com  /secure/success
83654  2008-07-22T06:04:24.902157000Z  192.168.15.4  69.25.94.22  www.willselfdestruct.com  /images/bk-message.gif

(kali@kali)-[~/SBT-DF204-CS1]
$
```

Fig 9: Identifying the client IP and the Destination IP

The Output shows that the Client IP is 192.168.15.4 and it denotes that it corresponds to a residence router while the Service IP is 69.25.94.22 which is the destination for every request as we filtered out in the query.

The GET query shows that the page was accessed within frames 82936, the pages loaded and then it was successfully submitted 83614.

These also showed that the site (www.willselfdestruct.com) was accessed on the 22 July in UTC and not on the 21st of July as noted on the brief.

Also running a DNS lookup to be able to ascertain the service IP

```
(kali@kali)-[~/SBT-DF204-CS1]
$ tshark -r working/nitroba_working.pcap -Y 'dns.qry.name contains "willselfdestruct"' \
-T fields -e frame.number -e frame.time_utc -e ip.src -e dns.qry.name -e dns.a
82912  2008-07-22T06:03:43.640466000Z  192.168.15.4  www.willselfdestruct.com
82923  2008-07-22T06:03:43.720123000Z  192.168.1.254  www.willselfdestruct.com  69.25.94.22,64.202.165.178,208.109.255.11

(kali@kali)-[~/SBT-DF204-CS1]
$
```

To return to your computer, move the mouse pointer outside or press Ctrl+Alt.

Fig 10: Running a DNS Query Lookup

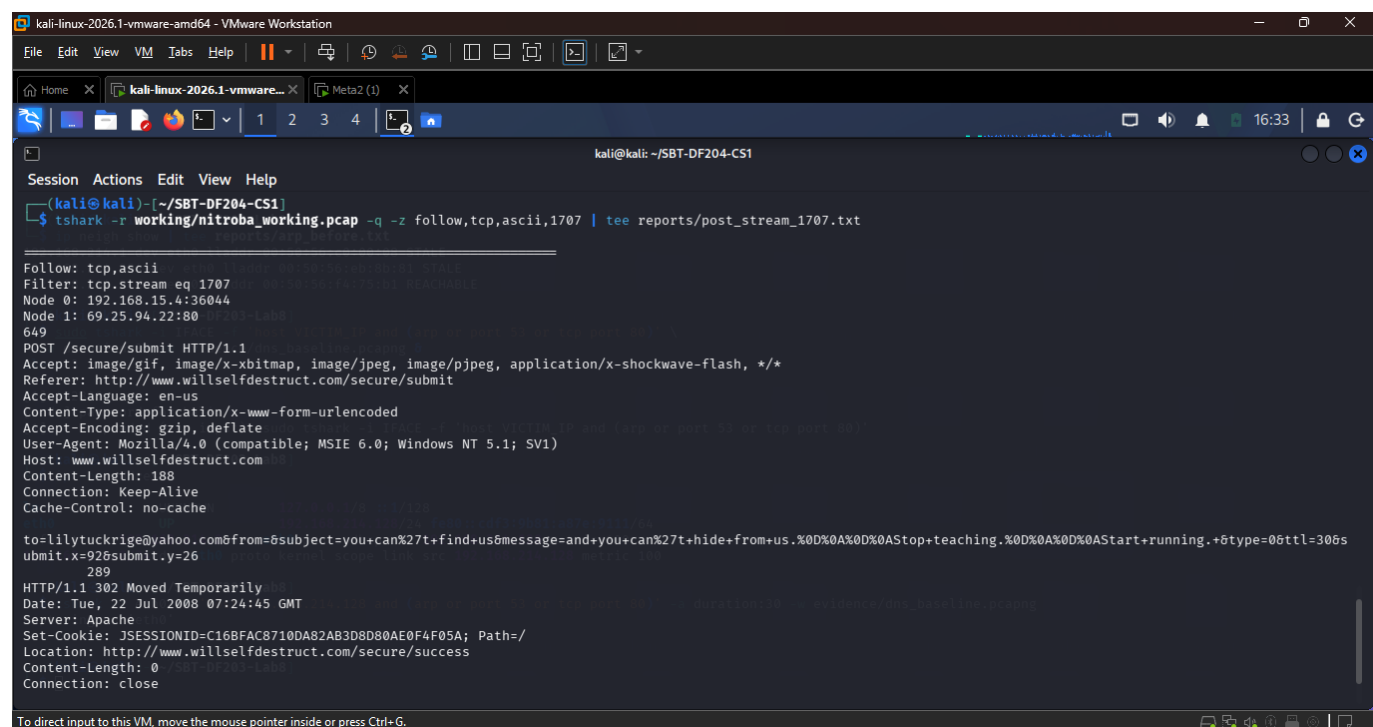
Step 3: To get the Message and link the client to the Message (POST Request)

The Next step was to get the POST request by using a POST filter Query, so we can link the client to the message that was sent.

```
(kali@kali)-[~/SBT-DF204-CS1]
$ tshark -r working/nitroba_working.pcap -Y 'http.request.method=="POST" && http.host contains "willselfdestruct"' \
-T fields -e frame.number -e frame.time_utc -e ip.src -e ip.dst -e tcp.stream -e http.request.uri
83601  2008-07-22T06:04:24.311700000Z  192.168.15.4  69.25.94.22  1707  /secure/submit
```

The capture shows that the client at 192.168.15.4 sent an HTTP POST to willselfdestruct.com (69.25.94.22) at 06:04:24 UTC on 22 July 2008. This is frame 83601 in TCP stream 1707, and the request was made to /secure/submit. About 40 seconds earlier, at frame 82936, the same client had opened that page. Shortly after the POST, at frame 83614, the client requested the /secure/success page, which suggests the service accepted the submission. This shows that a form was submitted from this client at that time. What the form contained is covered in the next finding.

So, I also explored the POST stream 1707 to be able to get detailed information from the capture



```
kali@kali: ~/SBT-DF204-CS1
$ tshark -r working/nitroba_working.pcap -q -z follow,tcp,ascii,1707 | tee reports/post_stream_1707.txt

Follow: tcp,ascii
Filter: tcp.stream eq 1707
Node 0: 192.168.15.4:36044
Node 1: 69.25.94.22:80
649
POST /secure/submit HTTP/1.1
Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, application/x-shockwave-flash, */*
Referer: http://www.willselfdestruct.com/secure/submit
Accept-Language: en-us
Content-Type: application/x-www-form-urlencoded
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; SV1)
Host: www.willselfdestruct.com
Content-Length: 188
Connection: Keep-Alive
Cache-Control: no-cache

to=lilytuckrige@yahoo.com&from=&subject=you+can%27t+find+us&message=and+you+can%27t+hide+from+us.%0D%0A%0D%0AStop+teaching.%0D%0A%0D%0AStart+running.+&type=0&ttl=306s
ubmit.x=926submit.y=26
289
HTTP/1.1 302 Moved Temporarily
Date: Tue, 22 Jul 2008 07:24:45 GMT
Server: Apache
Set-Cookie: JSESSIONID=C16BFAC8710DA82AB3D8D80AE0F4F05A; Path=/
Location: http://www.willselfdestruct.com/secure/success
Content-Length: 0
Connection: close
```

Fig 12: POST Stream '1707' exploration/analysis

The content of the POST in frame 83601 (stream 1707) was recovered with a follow TCP Stream. The form data showed that the message was addressed to lilytuckrige@yahoo.com with the subject "you can't find us" and a body reading "and you can't hide from us. Stop teaching. Start running."

The from field was empty, therefore we can't decipher who sent the message.

The service also answered with a 302 redirect to /secure/success, which the client then requested at frame 83614 as we see in Figure 9. The recipient and content correspond to the reported harassment message, subject to comparison with the second email in the case materials. This links the traffic from 192.168.15.4 to the message, but it doesn't identify who was using the device.

Step 4: Identifying the Device

Firstly, I had to identify the device behind the traffic, I started with the frame of the relevant POST request, frame 83601, which I had already identified in the earlier findings. Inspecting its Ethernet header showed a source MAC address of 00:17:f2:e2:c0:ce and a destination MAC address of 00:1d:d9:2e:4f:60 (Fig 13).

```
(kali㉿kali)-[~/SBT-DF204-CS1]
$ tshark -r working/nitroba_working.pcap -Y 'frame.number==83601' -T fields -e eth.src -e eth.dst
00:17:f2:e2:c0:ce      00:1d:d9:2e:4f:60

(kali㉿kali)-[~/SBT-DF204-CS1]
$
```

Fig 13: Source Mac Address Identification

```
(kali㉿kali)-[~/SBT-DF204-CS1]
$ tshark -r working/nitroba_working.pcap -Y 'ip.src==192.168.15.4' -T fields -e eth.src | sort | uniq -c
34554 00:17:f2:e2:c0:ce

(kali㉿kali)-[~/SBT-DF204-CS1]
$
```

Fig 14: Traffic sent from the Source IP to the Destination

```
(kali㉿kali)-[~/SBT-DF204-CS1]
$ tshark -r working/nitroba_working.pcap -Y 'ip.dst==192.168.15.4' -T fields -e eth.src | sort | uniq -c
38643 00:1d:d9:2e:4f:60

(kali㉿kali)-[~/SBT-DF204-CS1]
$
```

Fig 15: Traffic sent to the Source IP from the Destination

To check whether this MAC was consistent in Fig 13, I filtered all traffic sent from 192.168.15.4. All 34,554 frames carried the same source MAC, 00:17:f2:e2:c0:ce (Fig 14). I then filtered all traffic sent to 192.168.15.4. All 38,643 frames came from 00:1d:d9:2e:4f:60 (Fig 15). Because the POST was addressed to an external web server, this second MAC is the next-hop device, probably the gateway, and not the web server itself.

This shows that, within the capture, 192.168.15.4 was used by one network interface, 00:17:f2:e2:c0:ce, and that interface sent the POST to willselfdestruct.com. But we can't conclude with just an IP address as many devices can share one Public IP behind a router. The MAC is meant to be the stronger Indicator as to which machine sent the message as we can't determine the Attacker from the MAC address. Another Limitation is that the MAC Address can be changed or spoofed so it doesn't necessarily show who sent the message.

Step 5: Identity from Cookies

We have to identify the particular sites that the MAC Address has been in communication with and it is shown below.

```
(kali@kali)-[~/SBT-DF204-CS1]
$ tshark -r working/nitroba_working.pcap -Y 'eth.src==00:17:f2:e2:c0:ce && http.cookie contains "@" -T fields -e http.host | sort | uniq -c | sort -rn
  97 mail.google.com
  33 ad.yieldmanager.com
   9 www.google.com
   5 www.amazon.com
   2 b.mail.google.com
   2 adopt.specificclick.net
   1 chatenabled.mail.google.com

(kali@kali)-[~/SBT-DF204-CS1]
$
```

Fig 16:

Then we explore the one that is more conspicuous such as the mail.google.com that has 97 connections

```
1 chatenabled.mail.google.com

(kali@kali)-[~/SBT-DF204-CS1]
$ tshark -r working/nitroba_working.pcap -Y 'eth.src==00:17:f2:e2:c0:ce && http.host contains "mail.google.com" && http.cookie contains "@" -T fields -e http.cookie | grep -oE '[A-Za-z_-]+@[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}' | sort | uniq -c
  100 gmailchat=jcoachj@gmail.com

(kali@kali)-[~/SBT-DF204-CS1]
$ tshark -r working/nitroba_working.pcap -Y 'eth.src==00:17:f2:e2:c0:ce && http.host contains "mail.google.com" && http.cookie contains "@" -T fields -e frame.number -e frame.time_utc -e http.host | sort -n | sed -n '1p;$p'
78967 2008-07-22T06:01:02.113436000Z mail.google.com
84201 2008-07-22T06:04:55.380722000Z mail.google.com

(kali@kali)-[~/SBT-DF204-CS1]
$
```

Fig 17:

To look for identity evidence, I searched the HTTP requests sent from MAC 00:17:f2:e2:c0:ce for cookies containing "@". Most matches came from advertising and shopping sites such as Amazon and ad networks, and these contained encoded values and not email addresses, so I ruled them out. The Gmail hosts accounted for 100 matching requests, and every one carried the cookie gmailchat=jcoachj@gmail.com (Fig 17). These requests ran from frame 78967 (06:01:02 UTC) to frame 84201 (06:04:55 UTC), a period that includes the POST at frame 83601 (06:04:24 UTC) (Fig 18). This indicates that the device which sent the message was using a browser associated with the Gmail account jcoachj@gmail.com at that time. It does not show who was operating the device, since another person could have used the same browser or computer.

Class Roster is as seen below.

So who did it?

Chemistry 109 class list:

Teacher: Lily Tuckrige

Students:

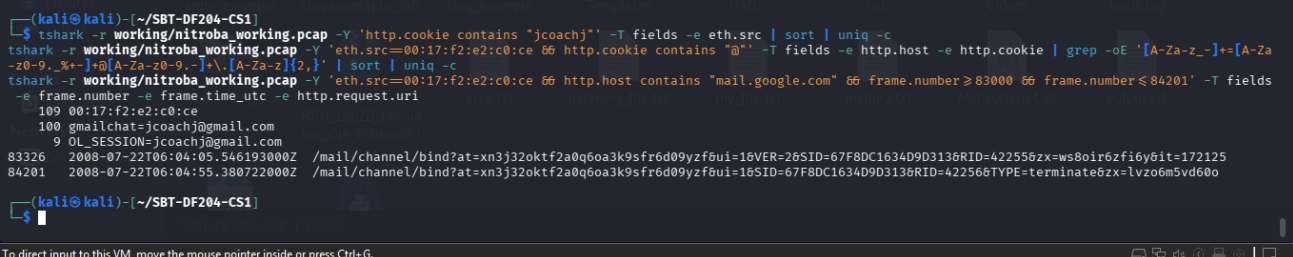
Amy Smith
Burt Greedom
Tuck Gorge
Ava Book
Johnny Coach
Jeremy Ledvkin
Nancy Colburne
Tamara Perkins
Esther Pringle
Asar Misrad

Fig 18: Students Chemistry 109 roster

From the evidence and the roster Johnny Coach is a direct link as inferred from the evidence as the **jcoachj** username is consistent with the student's name Johnny Coach.

The Gmail address recovered from the cookie evidence, jcoachj@gmail.com, was compared with the Chem 109 class roster provided in the scenario materials (Fig 19). The roster lists a student named Johnny Coach. The address contains the first initial and surname of this student, which suggests the account may belong to him.

I treat it as an inference and not a confirmed identification. Enrollment in the class shows he had a possible connection to the recipient of the message, but it does not show that he was the person using the device at the time of the POST.



```
(kali@kali)-[~/SBT-DF204-CS1]
$ tshark -r working/nitroba_working.pcap -Y 'http.cookie contains "jcoachj"' -T fields -e eth.src | sort | uniq -c
tshark -r working/nitroba_working.pcap -Y 'eth.src==00:17:f2:e2:c0:ce && http.cookie contains "@"' -T fields -e http.host -e http.cookie | grep -oE '[A-Za-z_-]+=[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}' | sort | uniq -c
tshark -r working/nitroba_working.pcap -Y 'eth.src==00:17:f2:e2:c0:ce && http.host contains "mail.google.com" && frame.number >= 83000 && frame.number <= 84201' -T fields -e frame.number -e frame.time_utc -e http.request.uri
109 00:17:f2:e2:c0:ce
100 gmailchat=jcoachj@gmail.com
9 OL_SESSION=jcoachj@gmail.com
83326 2008-07-22T06:04:05.546193000Z /mail/channel/bind?at=xn3j32oktf2a0q6oa3k9sfr6d09y2f6ui=16VER=26SID=67F8DC1634D9D3136RID=422556zx=ws8oir6zfi6y6it=172125
84201 2008-07-22T06:04:55.380722000Z /mail/channel/bind?at=xn3j32oktf2a0q6oa3k9sfr6d09y2f6ui=16SID=67F8DC1634D9D3136RID=422566TYPE=terminate6zx=lvzo6m5vd60o
(kali@kali)-[~/SBT-DF204-CS1]
$
```

Fig 19: Confirming if one device sent the mail and is using the MAC address

To check whether the Gmail cookie was specific to one device, I thoroughly searched the whole capture for cookies containing "jcoachj".

All 109 matching frames came from the MAC address 00:17:f2:e2:c0:ce, the same device that sent the POST (Fig 16). Searching that device's cookies for email addresses returned only jcoachj@gmail.com, in the gmailchat cookie (100 times) and the OL_SESSION cookie (9 times), so no other address was linked to it. Gmail requests from this device ran from 06:01:02 to 06:04:55 UTC. This indicates that the Gmail account jcoachj@gmail.com was active in a browser on the device at the time of the submission. This still does not establish who was using the device but it points strongly at Johnny Coach as he is the owner of the gmail account.

Step 6:

Time (UTC)	Frame	Event
06:01:02.113	78967	First Gmail request with jcoachj cookie
06:03:43.640	82912	DNS query for www.willselfdestruct.com
06:03:43.826	82936	GET /secure/submit (form page loaded)
06:04:05.546	83326	Gmail channel bind request
06:04:24.312	83601	POST to /secure/submit
06:04:24.564	83614	GET /secure/success
06:04:55.381	84201	Gmail channel terminate request

WIRESHARK ANALYSIS

Wireshark - Protocol Hierarchy Statistics - nitroba_working.pcap

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits
▼ Frame	100.0	94410	100.0	54670237	27
▼ Ethernet	100.0	94410	3.3	1822854	92
▼ Logical-Link Control	0.0	5	0.0	20	0
Data	0.0	5	0.0	10	0
▼ Internet Protocol Version 4	95.7	90379	3.3	1807760	91
▼ User Datagram Protocol	7.6	7200	0.1	57600	29
▼ Transmission Control Protocol	88.0	83114	4.6	2494088	1,2
Yahoo YMSG Messenger Protocol	0.0	8	0.0	1659	0
XMPP Protocol	0.0	18	0.0	2015	1
VSS Monitoring Ethernet trailer	0.0	1	0.0	8	0
Transport Layer Security	1.5	1408	3.8	2089117	1,0
Parallel Redundancy Protocol (IEC62439 Part 3)	0.0	3	0.0	12	0
MSN Messenger Service	0.0	6	0.0	400	0
Malformed Packet	0.0	7	0.0	0	0
▼ Hypertext Transfer Protocol	10.6	9965	25.2	13754386	6,9
Portable Network Graphics	0.1	136	1.3	712553	36
Online Certificate Status Protocol	0.0	2	0.0	1594	0
Media Type	0.5	467	14.9	8137365	4,1
Line-based text data	0.8	744	29.0	15836148	8,0
JPEG File Interchange Format	1.2	1180	18.4	10077905	5,1
HTML Form URL Encoded	0.0	28	0.0	7955	4
eXtensible Markup Language	0.6	581	4.8	2608205	1,3
CompuServe GIF	1.2	1136	3.0	1641272	83
Action Message Format	0.0	1	0.0	33	0
Data	0.1	57	0.4	239763	121

No display filter.

Close Copy Protocols Help

Fig 20: Protocol Hierachy for Nitrocap.pcap on wireshark

nitroba_working.pcap CPU usage: 7.8%

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.64	74.125.19.83	TCP	70	42760 → 80 [FIN, ACK] Seq=1 Ack=1 Win=65535 Len=0 TSval=712729540 TSecr=980517270
2	0.008450	74.125.19.83	192.168.1.64	TCP	70	80 → 42760 [FIN, ACK] Seq=1 Ack=2 Win=431 Len=0 TSval=980523165 TSecr=712729540
3	0.019619	192.168.1.64	74.125.19.19	HTTP	1421	GET /mail/?logout&hl=en HTTP/1.1
4	0.044170	74.125.19.19	192.168.1.64	TCP	70	80 → 35011 [ACK] Seq=1 Ack=1352 Win=393 Len=0 TSval=988915614 TSecr=712729541
5	0.224402	74.125.19.19	192.168.1.64	HTTP	1284	HTTP/1.1 302 Moved Temporarily
6	0.226712	192.168.1.64	74.125.19.19	TCP	70	35011 → 80 [ACK] Seq=1352 Ack=1215 Win=65460 Len=0 TSval=712729543 TSecr=988915792
7	0.231239	192.168.1.64	74.125.19.19	TCP	70	35011 → 80 [FIN, ACK] Seq=1352 Ack=1215 Win=65535 Len=0 TSval=712729543 TSecr=988915792
8	0.240276	74.125.19.19	192.168.1.64	TCP	70	80 → 35011 [FIN, ACK] Seq=1215 Ack=1353 Win=393 Len=0 TSval=988915810 TSecr=712729543
9	0.280893	192.168.1.64	192.168.1.254	DNS	78	Standard query 0x61ce A www.google.com
10	0.283114	192.168.1.64	192.168.1.254	DNS	78	Standard query 0x7362 AAAA www.google.com
11	0.294021	192.168.1.254	192.168.1.64	DNS	386	Standard query response 0x61ce A www.google.com CNAME www.l.google.com A 74.125.19.103
12	0.295200	192.168.1.254	192.168.1.64	DNS	78	Standard query response 0x7362 AAAA www.google.com
13	0.308778	192.168.1.64	192.168.1.254	DNS	80	Standard query 0xf689 AAAA www.l.google.com
14	0.321240	192.168.1.254	192.168.1.64	DNS	80	Standard query response 0xf689 AAAA www.l.google.com
15	0.328385	192.168.1.64	74.125.19.103	TCP	82	39153 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1452 WS=0 TSval=712729544 TSecr=0 SACK_PERM
16	0.337092	74.125.19.103	192.168.1.64	TCP	78	443 → 39153 [SYN, ACK] Seq=0 Ack=1 Win=5672 Len=0 MSS=1408 SACK_PERM TSval=997777432 TSecr=712729544
17	0.339453	74.125.19.83	192.168.1.64	TCP	70	[TCP Retransmission] 80 → 42760 [FIN, ACK] Seq=1 Ack=2 Win=431 Len=0 TSval=980523497 TSecr=712729541
18	0.340872	192.168.1.64	74.125.19.103	TCP	70	39153 → 443 [ACK] Seq=1 Ack=1 Win=524280 Len=0 TSval=712729544 TSecr=997777432
19	0.341468	192.168.1.64	74.125.19.83	TCP	70	42760 → 80 [ACK] Seq=2 Ack=2 Win=65535 Len=0 TSval=712729544 TSecr=980523497

Frame 1: Packet, 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface 0

Ethernet II, Src: HonHaiPrecis_2e:4f:61 (00:1d:d9:2e:4f:61), Dst: Commscope_99:98:61 (00:1d:d9:99:98:61)

Internet Protocol Version 4, Src: 192.168.1.64, Dst: 74.125.19.83

Transmission Control Protocol, Src Port: 42760, Dst Port: 80, Seq: 1, Ack: 1, Len: 0

0000 00 1d b5 99 98 68 00 1d d9 2e 4f 61 08 00 45 00 ...k...h...
0010 00 34 68 4e 40 00 3f 06 b3 bd c0 a8 01 40 4a 7d ...4hN0...
0020 13 53 a7 08 00 50 fd fa 92 12 b5 24 f8 f1 80 11 ...S...P...
0030 ff ff 27 40 00 00 01 01 08 0a 2a 7b 63 c4 3a 71 ...'@...
0040 81 96 9d cc ba 1e ...

nitroba_working.pcap Packets: 94410 Profile: Default

Fig 21:

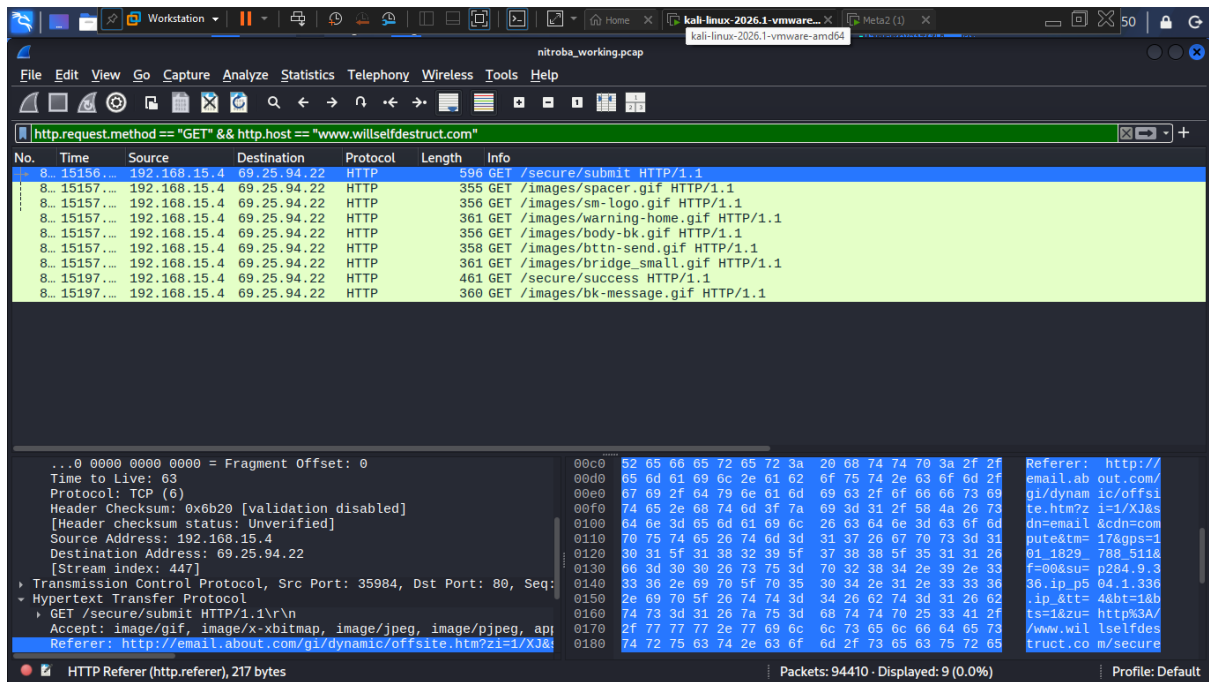


Fig 22: Applying Filter

Identifying The Client IP and the Destination IP

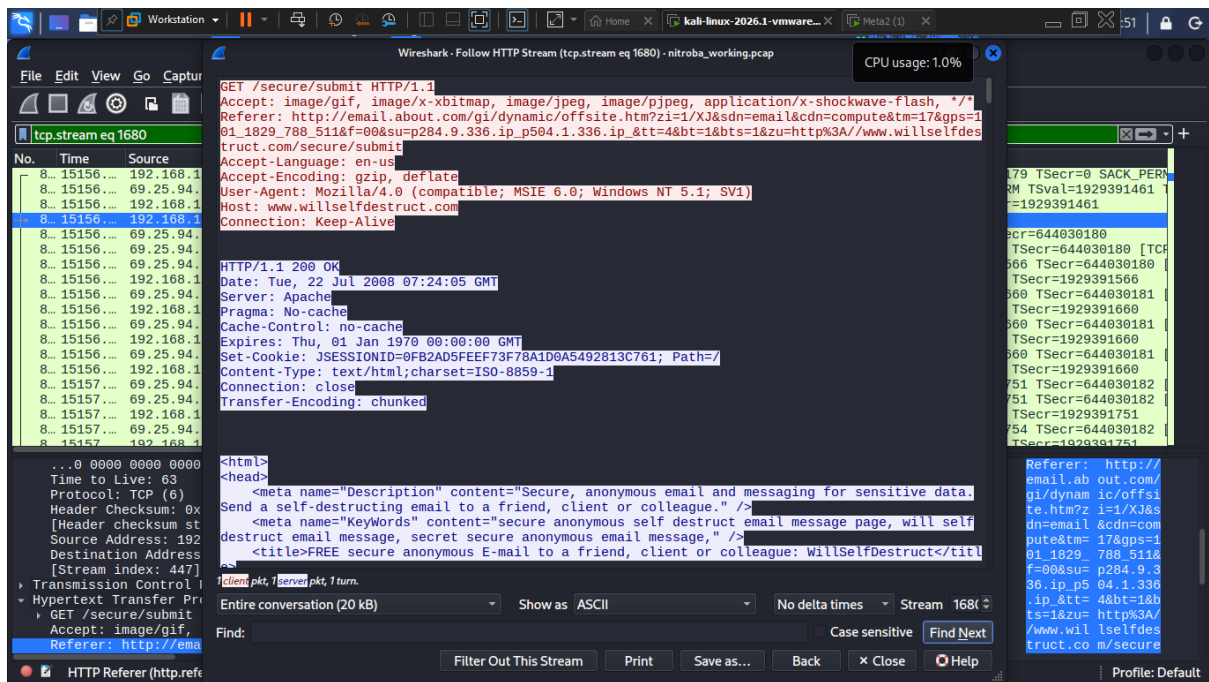


Fig 23: Following HTTP Stream

Evidence Log

Evidence ID	Packet or item	Finding	Why it matters	Screenshot or appendix
E01	nitroba.pcap (Original and Working Copy)	Original preserved read-only, with a working copy	Shows the evidence was preserved and the analysis was repeatable.	Fig 3
E02	Frames 82912 and 82923 (DNS), frame 82936 (GET /secure/submit), 22 July 2008, 06:03:43 UTC	Client 192.168.15.4 queried www.willselfdestruct.com Destination: 69.25.94.22	Establishes the client, the service and the start of the session.	Fig 5
E03	Frame 83601, stream 1707, 06:04:24.311700 UTC; frame 83614 (GET /secure/success), 06:04:24.564165 UTC	HTTP POST from 192.168.15.4 to 69.25.94.22, /secure/submit. The server replied with a 302 redirect to /secure/success, which the client then requested.	Shows a form submission from this client to the service at this time.	Fig 11
E04	Stream 1707 (Follow TCP Stream)	Form data: to = lilytuckrige@yahoo.com , User-Agent MSIE 6.0 on Windows NT 5.1.	Links the traffic to the harassment message content.	Fig 12
E05	Ethernet Header (Frame 83601)	Client MAC 00:17:f2:e2:c0:ce on all 34,554 frames from the IP.	Ties the POST to one network interface, which is more specific than an IP address. It identifies a device, not a person.	Fig 13, 14, 15

Evidence ID	Packet or item	Finding	Why it matters	Screenshot or appendix
E06	Frames 78967 to 84201 (Gmail requests)	The cookie jcoachj@gmail.com appeared in 109 frames, all from MAC 00:17:f2:e2:c0:ce (gmailchat 100, OL_SESSION 9). No other email-shaped cookie was sent by that device, and no other device sent this cookie. Gmail activity runs from 06:01:02 to 06:04:55 UTC and includes the POST.	It associates the sending device's browser with a named Gmail account	Fig 17

Conclusion

Summary of findings

The capture shows that on 22 July 2008 a device at 192.168.15.4 (MAC 00:17:f2:e2:c0:ce) loaded the willselfdestruct.com compose page, submitted a message, and received the service's confirmation. The submitted message was addressed to lilytuckrige@yahoo.com, had the subject "you can't find us", and read "and you can't hide from us. Stop teaching. Start running." and it corresponds to the second email described in the case materials. The device was also running a Gmail session for the account jcoachj@gmail.com at the time and it was the same device. That account name is consistent with Johnny Coach, a student on the Chem 109 roster.

Things I Observed were that

- The Reported harassment message after a packet sniffer was placed on the network corresponds with the findings from the forensic Analysis and the from field was empty.
- The only email address in cookies sent by this device was jcoachj@gmail.com (109 frames). No other device in the capture sent this cookie.
- The User-Agent was MSIE 6.0 on Windows NT 5.1.
- The Period of which the Gmail was actively running was also the time frame the POST request of the email was dropped.

What can be drawn out from this Analysis:

1. Definitely, we know that the message was sent from the device identified by MAC 00:17:f2:e2:c0:ce.
2. The Gmail account was opened in a web browser. The account name is consistent with the roster entry for Johnny Coach, which suggests he may be connected to the device.

Limitations:

We can only say a particular device carried it out and it can only give us calculated and factual Inferences as we have in this analysis. But we can't say a Particular person carried out the attack. Johnny Coach name on the roster is just consistent and very similar with the email associated with the harassment.

Cookies show presence, not use. The Gmail cookies show an active session, not that the account was used to send this message, which went through a web form to a Yahoo address.

CONFIDENCE AND RECOMMENDATION

Therefore, from the adequate analysis carried out I have high confidence that the message was submitted from the device at 192.168.15.4 with MAC 00:17:f2:e2:c0:ce, because the POST, the content and the consistent MAC all come directly from the capture. I have moderate confidence that this device was associated with the account jcoachj@gmail.com, because it rests on a cookie seen consistently across the capture. This leads to Johnny Coach as a person involved which is a lead and not a conclusion, since it depends on a name resemblance and on assumptions about who used the device. I'd recommend that investigators confirm the device ownership, interview the account holder, and examine the device itself before drawing any conclusion about the person.